

Universitatea din București
Facultatea de Matematică și Informatică
Departamentul Informatică
Specializarea Calculatoare și Tehnologia Informației

Trippi

Asistent de călătorii bazat pe inteligență artificială

Documentația proiectului

Coordonatori:

Horațiu Cheval

Ciprian Ceaușescu

Studenti:

Matei Pavel

Burcă-Andonie Daniel

Nica Răzvan

Cuprins

Cuprins.....	Error! Bookmark not defined.
1. Introducere	4
2. Cerințele proiectului	4
3. Funcționalități principale	4
4. Arhitectura aplicației	4
4.1 Stack tehnologic	4
4.2 Aplicații Django	5
4.3 Subsistemul AI (Facade).....	5
5. Agenții AI	5
5.1 Concierge (Agentul 1)	5
5.2 Data Architect (Agentul 2).....	5
5.3 Tool Planner și uneltele	5
6. Uneltele de date reale	6
7. Design patterns folosite	6
8. Integrări externe (API-uri)	6
9. Fluxul unei conversații	6
10. Tratarea erorilor și reziliența	7
11. Testarea automată.....	7
12. Bug-uri găsite și rezolvate (raportare și fix)	7
12.1 An greșit la planificarea zborurilor	7
12.2 Zboruri cu escală afișate ca directe.....	7
12.3 Mesaj de eroare dublu care ascundea cauza reală	8
12.4 Preferință falsă pentru hoteluri de 3 stele.....	8
12.5 Conversația se rupea la răspunsul scurt	8
12.6 Orașe scrise în română, nerecunoscute de API-uri.....	8

12.7 Linkul Google Flights ducea la pagina principală.....	8
12.8 Linkuri de hotel greșite sau inventate (404).....	9
13. Raport despre folosirea instrumentelor AI în dezvoltare.....	9
13.1 Instrumente folosite	9
13.2 Unde a ajutat AI-ul (cazuri concrete)	9
13.3 Ce nu a mers / limitări observate	9
13.4 Verificarea rezultatelor	10
13.5 Concluzie.....	10
14. Diagrame (Mermaid)	10
14.1 Arhitectura componentelor.....	10
14.2 Fluxul unei conversații (secvență)	11
14.3 Diagrama de clase.....	12
15. Concluzii	12

1. Introducere

Trippi este un asistent de călătorii bazat pe inteligență artificială: un chat conversațional care planifică călătorii (itinerarii, cazare, buget, transport) și învață în mod tăcut preferințele utilizatorului din conversație. Aplicația este construită cu Django 6 și MySQL, iar partea de AI folosește modele Google Gemini și surse de date reale (zboruri, hoteluri și direcții).

Documentul descrie cerințele, arhitectura, agenții AI, șabloanele de proiectare, integrările externe, fluxul unei conversații, tratarea erorilor, bug-urile rezolvate, testarea automată, folosirea AI și diagramele.

2. Cerințele proiectului

Proiectul cere o aplicație cu minim doi agenți AI (prin apeluri API sau modele locale), cu accent pe procesul de dezvoltare: specificații și backlog, arhitectură și diagrame, control al versiunilor cu Git (branch-uri, merge, rebase, pull request-uri), teste automate, raportare și rezolvare de bug-uri, design patterns și un raport despre folosirea instrumentelor AI în dezvoltare.

Trippi acoperă partea de implementare prin doi agenți AI principali (Concierge și Data Architect) plus un planificator de unelte și trei unelte de date reale.

3. Funcționalități principale

- **Asistent conversațional:** chat conversațional în limba română, cu istoric pe sesiuni (creare, redenumire, ștergere).
- **Planificare:** sugestii de itinerarii, cartiere, buget orientativ și activități.
- **Preferințe învățate:** preferințele (buget, ritm, stele hotel, interese, restricții) sunt extrase automat din conversație și reutilizate.
- **Zboruri în timp real:** căutare reală de zboruri, cu marcarea clară direct / cu escală și locul escalei.
- **Hoteluri în timp real:** căutare reală de cazare cu prețuri și scoruri.
- **Direcții:** rute reale (transport public, mașină, mers pe jos) între orice două locații, cu pași și link Google Maps live.

4. Arhitectura aplicației

4.1 Stack tehnologic

- **Backend:** Django 6, Python 3.12.
- **Bază de date:** MySQL.

- **Modele AI:** Google Gemini (gemini-2.5-flash-lite).
- **Date reale:** RapidAPI (google-flights2, booking-com18) și Google Maps Directions API.
- **Frontend:** HTML, CSS (temă travel, verde) și JavaScript (AJAX, randare Markdown).

4.2 Aplicații Django

- **users:** autentificare, profil și modelul UserPreference.
- **trips:** modelele ChatSession și ChatMessage, view-urile de chat și operațiile pe sesiuni.
- **agents:** tot subsistemul de inteligență artificială (agenți, planner, unelte, client LLM).

4.3 Subsistemul AI (Facade)

Întreaga logică AI stă în spatele unei singure funcții, `orchestrator.handle_user_message`, astfel încât view-urile din trips nu depind de detaliile providerului. Orchestratorul încarcă preferințele și istoricul, rulează agentul conversațional, apoi pornește în fundal agentul de extragere a preferințelor.

5. Agenții AI

5.1 Concierge (Agentul 1)

Asistentul conversațional, vizibil pentru utilizator. Își încarcă promptul de sistem, îl personalizează cu preferințele utilizatorului, transformă istoricul în formatul providerului și delegă apelul către un client LLM. Înainte de a răspunde, verifică dacă mesajul necesită date reale și, dacă da, le obține și ancorează răspunsul în ele.

5.2 Data Architect (Agentul 2)

Agent tăcut de extragere a preferințelor. Citește conversația, cere modelului un JSON strict conform schemei UserPreference, apoi îl validează și îl fuzionează conservator în preferințele salvate. Rulează în fundal (thread separat), ca să nu întârzie răspunsul Concierge-ului.

5.3 Tool Planner și unelte

Un al treilea pas AI decide dacă mesajul cere zboruri, hoteluri sau direcții și extrage parametrii ca JSON strict. Un filtru ieftin pe cuvinte-cheie evită apelarea modelului pentru mesaje irelevante.

6. Unelele de date reale

- **Zboruri:** rezolvă orașul în cod IATA (searchAirport) apoi caută zboruri (google-flights2). Marchează corect direct/escală folosind câmpul layovers. Linkul duce la pagina de rezultate Google Flights (parametrul tfs).
- **Hoteluri:** auto-complete pentru locație apoi căutare de cazare (booking-com18). Linkul este pagina reală Booking a hotelului.
- **Direcții:** Google Maps Directions API; întoarce un link live și, când cheia e configurată, narează ruta în chat.

7. Design patterns folosite

- **Strategy:** clasa LLMClient cu implementări interschimbabile (GeminiClient, EchoClient offline).
- **Factory:** make_llm_client construiește clientul potrivit din configurație și cade elegant pe EchoClient.
- **Facade:** orchestrator.handle_user_message ascunde tot subsistemul de agenți după un singur apel.
- **Command / Tool:** fiecare capabilitate externă (zboruri, hoteluri, direcții) e un Tool cu interfață uniformă.

8. Integrări externe (API-uri)

- **LLM:** Google Gemini pentru Concierge, Data Architect și planner.
- **Zboruri:** google-flights2 (RapidAPI).
- **Hoteluri:** booking-com18 (RapidAPI).
- **Direcții:** Google Maps Directions API.

Toate cheile se citesc din variabile de mediu (.env, exclus din Git), nu sunt hardcodate.

9. Fluxul unei conversații

1. Utilizatorul trimite un mesaj; view-ul salvează turul în baza de date.
2. Orchestratorul încarcă preferințele și ultimele mesaje și apelează Concierge-ul.
3. Planner-ul decide dacă e nevoie de o unealtă și extrage parametrii (cu data curentă, pentru anul corect).
4. Dacă e cazul, unealta aduce date reale, care se injectează în prompt ca secțiunea DATE REALE.
5. Concierge-ul (Gemini) generează răspunsul, ancorat în datele reale.

6. În fundal, Data Architect-ul extrage și salvează preferințele noi.

10. Tratarea erorilor și reziliența

- **Fallback:** o eroare a Concierge-ului întoarce un mesaj prietenos; un eșec la cotă afișează un mesaj specific.
- **Best-effort:** eșecul agentului de preferințe este înghițit, ca să nu strice un răspuns bun.
- **Retry:** rezolvarea aeroporturilor reîncearcă o dată, fiindcă API-ul de zboruri e uneori lent.
- **Robustețe:** planner-ul și uneltele nu aruncă niciodată excepții către apelant.

11. Testarea automată

Proiectul are o suită de teste automate (Django, rulate cu comanda `python manage.py test`). Testele rulează complet offline: modelul AI real este înlocuit cu un client fals/echo, deci nu consumă cotă, sunt deterministe și prind regresii instant.

- **Factory / client:** alegerea providerului LLM și fallback-ul offline.
- **Agenți:** Concierge, planner, orchestrator (flux + erori), Data Architect (validare/merge).
- **Unelte:** hoteluri, helper-ul HTTP RapidAPI, uneltele de direcții.

12. Bug-uri găsite și rezolvate (raportare și fix)

Pe parcursul dezvoltării și al testării au fost identificate mai multe bug-uri reale, fiecare raportat și rezolvat printr-un fix dedicat (commit/pull request). Pentru fiecare se descriu simptomul, cauza și soluția.

12.1 An greșit la planificarea zborurilor

- **Simptom:** La întrebări de tipul zboruri pe 19 iunie, asistentul nu găsea zboruri și răspundea că nu are date în timp real.
- **Cauză:** Modelul nu cunoștea data curentă și deducea un an din trecut (2025). O dată trecută întoarce zero rezultate.
- **Soluție:** Injectarea datei curente în promptul planner-ului, plus o regulă explicită de an.
- **Fișier modificat:** `agents/planner.py`

12.2 Zboruri cu escală afișate ca directe

- **Simptom:** Zboruri care aveau de fapt o escală erau prezentate în chat ca fiind directe.

- **Cauză:** Câmpul stops din răspunsul API-ului era nefiabil (întorcea 0 chiar și pentru zboruri cu escală).
- **Soluție:** Determinarea direct/escală din câmpul layovers, cu orașul și durata escalei, marcate ca DIRECT sau CU ESCALĂ.
- **Fișier modificat:** agents/tools/flights.py

12.3 Mesaj de eroare dublu care ascundea cauza reală

- **Simptom:** La depășirea cotei, utilizatorul vedea un mesaj generic în loc de mesajul specific de limită.
- **Cauză:** Un try/except redundant în Concierge ascundea handler-ul mai bun din orchestrator.
- **Soluție:** Eliminarea try/except-ului redundant, lăsând orchestratorul să trateze unitar erorile providerului.
- **Fișier modificat:** agents/concierge.py

12.4 Preferință falsă pentru hoteluri de 3 stele

- **Simptom:** Utilizatorii noi apăreau ca având preferința de hotel de 3 stele, deși nu o exprimaseră.
- **Cauză:** Câmpul hotel_stars avea valoare implicită 3 și nu permitea NULL.
- **Soluție:** Câmpul a devenit nullable, fără valoare implicită, plus o migrare de bază de date.
- **Fișier modificat:** users/models.py

12.5 Conversația se rupea la răspunsul scurt

- **Simptom:** După ce asistentul cerea orașul de plecare, răspunsul scurt (din Stockholm) nu mai declanșa căutarea.
- **Cauză:** Filtrul de cuvinte-cheie verifica doar ultimul mesaj al utilizatorului.
- **Soluție:** Filtrul verifică acum și ultimul mesaj al asistentului.
- **Fișier modificat:** agents/planner.py

12.6 Orașe scrise în română, nerecunoscute de API-uri

- **Simptom:** Pentru unele orașe în română (Budapesta) nu apărea niciun zbor sau hotel.
- **Cauză:** Autocomplete-ul Google/Booking recunoaște doar forma internațională (Budapest).
- **Soluție:** Planner-ul scrie acum numele orașelor în engleză / forma internațională.
- **Fișier modificat:** agents/planner.py, agents/prompts/tool_planner.md

12.7 Linkul Google Flights ducea la pagina principală

- **Simptom:** Click pe linkul unui zbor deschidea homepage-ul Google Flights, nu rezultatele.

- **Cauză:** Linkul folosea o căutare text (q=); pagina de rezultate are nevoie de parametrul tfs.
- **Soluție:** Generarea linkului cu parametrul tfs, folosind ID-urile de entitate din searchAirport.
- **Fișier modificat:** agents/tools/flights.py

12.8 Linkuri de hotel greșite sau inventate (404)

- **Simptom:** Linkurile către hoteluri duceau la o pagină inexistentă, uneori cu hoteluri inventate.
- **Cauză:** Când unealta nu rula, modelul inventa linkuri; când rula, linkul era doar o căutare.
- **Soluție:** URL-ul real din endpoint-ul /stays/detail (în paralel) + regulă în prompt contra linkurilor inventate.
- **Fișier modificat:** agents/tools/hotels.py, agents/prompts/concierge.md

Toate fix-urile au fost validate prin suita de teste automate (52 de teste).

13. Raport despre folosirea instrumentelor AI în dezvoltare

Pe parcursul dezvoltării am folosit instrumente AI (asistenți de programare bazați pe modele mari de limbaj) ca partener de lucru, nu ca înlocuitor al deciziilor tehnice.

13.1 Instrumente folosite

- **Cod:** asistent AI de programare în editor, pentru generare de cod, refactorizare și depanare.
- **Documentație:** același asistent pentru generarea documentației și a rapoartelor.
- **În aplicație:** modelele Google Gemini, integrate în produs ca parte din agenți.

13.2 Unde a ajutat AI-ul (cazuri concrete)

- **Structură:** schelăria agenților și a uneltelor, urmând design pattern-uri clare.
- **Integrări:** integrarea și testarea API-urilor externe (Gemini, google-flights2, booking-com18, Google Maps).
- **Depanare:** identificarea și repararea bug-urilor reale din capitolul 12.
- **Probleme dificile:** decodarea formatului tfs al Google Flights (reverse engineering al unui protobuf).
- **Teste și documentație:** scrierea testelor automate și a documentației tehnice.

13.3 Ce nu a mers / limitări observate

- **Lipsa contextului temporal:** AI-ul nu cunoștea data curentă și deducea ani greșiți.
- **Erori subtile:** a generat ocazional erori subtile, corectate iterativ.

- **Presupuneri:** s-a bazat pe presupuneri despre API-uri care s-au dovedit greșite, descoperite prin testare.
- **Necesită verificare:** fără verificare umană, codul generat putea conține bug-uri funcționale.

13.4 Verificarea rezultatelor

- **Teste:** fiecare modificare a fost validată prin cele 52 de teste automate.
- **Testare manuală:** funcționalitățile au fost testate manual în chat.
- **Inspecție:** răspunsurile reale de la API-uri au fost inspectate înainte de folosire.

13.5 Concluzie

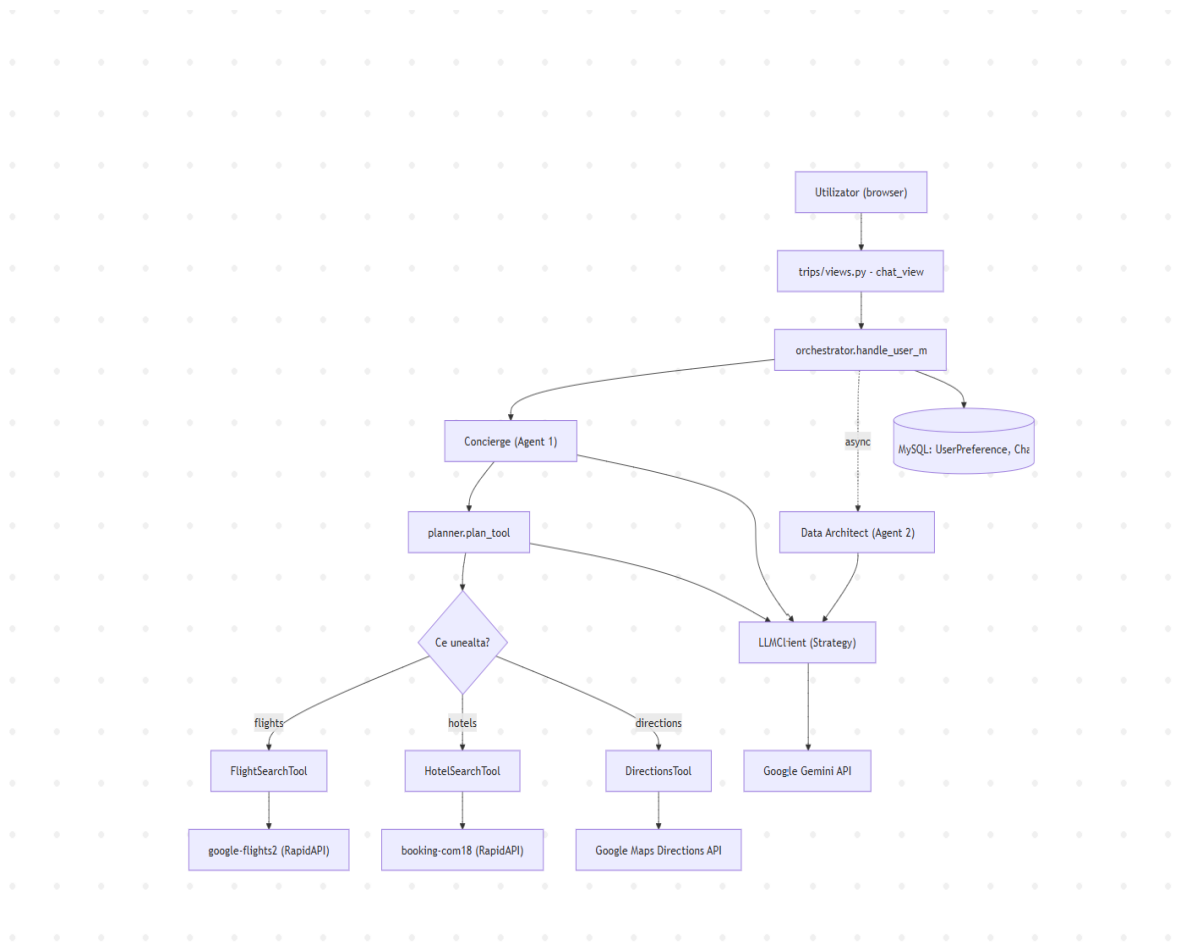
Instrumentele AI au accelerat dezvoltarea, mai ales la integrări, depanare și documentație, dar au fost cel mai utile ca asistent supravegheat: rezultatele au necesitat verificare, iar deciziile finale au rămas la echipă.

14. Diagrame (Mermaid)

Diagramele sunt scrise în format Mermaid. Codul de mai jos poate fi randat ca imagine pe <https://mermaid.live> (Copy-paste în editor).

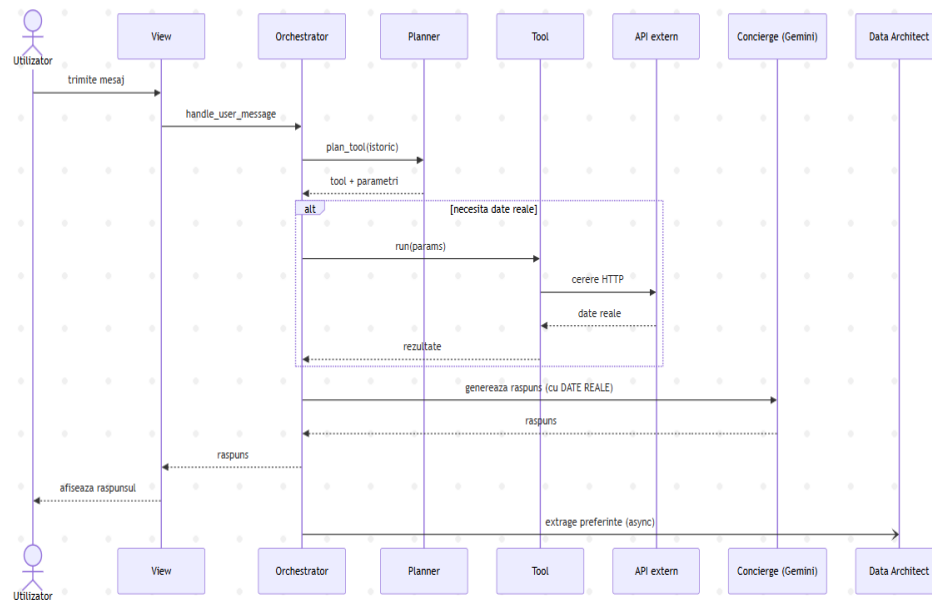
14.1 Arhitectura componentelor

Componentele aplicației și fluxul de apeluri către agenți, unelte și API-uri externe.



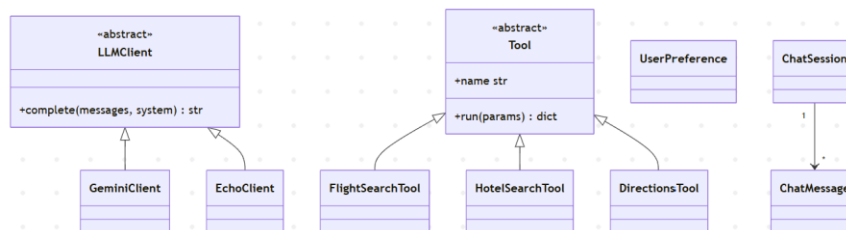
14.2 Fluxul unei conversații (secvență)

Pașii unui mesaj, de la utilizator până la răspuns, inclusiv extragerea preferințelor în fundal.



14.3 Diagrama de clase

Ierarhiile principale: clienții LLM (Strategy), uneltele (Command/Tool) și modelele de date.



15. Concluzii

Trippi demonstrează o arhitectură curată, orientată pe agenți, în care logica AI este izolată în spatele unui Facade și construită din componente interschimbabile (Strategy/Factory). Aplicația combină un model conversațional cu surse de date reale, tratează elegant erorile, a trecut printr-un proces de identificare și rezolvare de bug-uri și este acoperită de teste automate.